Trabajo Practico N° 4 – Programación 4

Para esta nueva versión del práctico, deberá desarrollar un **Backend con una API REST** que permita almacenar y recuperar participantes desde una base de datos (SQL o NoSQL).

La comunicación entre frontend (React) y backend deberá realizarse mediante **JSON**.

El backend deberá exponer los siguientes endpoints:

1. Obtener participantes

GET /participantes

Retorna la lista completa de participantes en formato JSON.

---

2. Crear participante

POST /participantes

Recibe un participante en JSON y lo guarda en la base de datos.

---

3. Eliminar participante

DELETE /participantes/{id}

Elimina un participante por ID.

El frontend deberá:

- Reemplazar el uso de localStorage
- Consumir la API mediante:
  - fetch o axios

Context API (useContext)

useContext

"¿Qué pasa si tengo muchos componentes y no quiero pasar props por todos lados?"

Entonces aparece Context

"Context permite compartir datos entre muchos componentes sin tener que pasar props manualmente."

Qué problema soluciona

Esto:

App → Padre → Hijo → Nieto → Bisnieto

Pasando props todo el tiempo

Con Context:

App (contexto global)
↓
Todos los componentes acceden directo

Mucho más limpio

Crear contexto

createContext()

Provider

<MiContext.Provider value={...}>

Consumir contexto

useContext(MiContext)

Objetivo del Práctico

Sacar participantes de Home y Llevarlo a un **Context**

Resultado:

Cualquier componente puede:

- Agregar participantes
- Eliminar
- Leer lista

Estructura Actualizada

```
src/
 ├── context/
 │    └── ParticipantesContext.tsx
 ├── components/
 │    ├── Formulario.tsx
 │    ├── Filtros.tsx
 │    └── ParticipanteCard.tsx
 ├── models/
 │    └── Participante.ts
 ├── Home.tsx
```

main.tsx

```
import { ParticipantesProvider } from "./context/ParticipantesContext";

<ParticipantesProvider>
  <Home />
</ParticipantesProvider>
```

ParticipanteContext

Deberá compartir en el contexto de la aplicación la variable participantes y sus correspondientes métodos de actualización.

```
interface ContextType {
  participantes: Participante[];
  agregar: (p: Participante) => void;
  eliminar: (id: number) => void;
  resetear: () => void;
}
```

Resultado Final

La aplicación sigue funcionando de la misma manera que la versión anterior pero ahora tenemos persistencia en base de datos y compartimos variable y métodos de forma general mediante el hook **useContext**